Improving RUNG: Adaptive Thresholding and Learnable Cosine Distance for Robust Graph Neural Networks

Tawkir Aziz Rahman*
Bangladesh University of Engineering and Technology
Dhaka, Bangladesh

Dipanta Kumar Roy Nobo†
Bangladesh University of Engineering and Technology
Dhaka, Bangladesh

Abstract

This report describes our project on improving RUNG, a robust Graph Neural Network published at NeurIPS 2024. RUNG uses a non-convex penalty called MCP to remove suspicious edges during message passing. It works well, but it has a clear weakness: its edge-suspiciousness threshold is fixed before training and stays fixed across all layers. When an attacker changes the attack budget, or when features smooth out in deeper layers, that fixed threshold becomes wrong. We built two improvements on top of the RUNG model. The first is a Percentile Gamma system that computes the threshold from real edge-distance statistics at each layer. The second is a Learnable Cosine Distance module that replaces the Euclidean distance used in RUNG with cosine similarity, and allows the network to learn the best geometric space to compare nodes. We test both methods on six datasets: Cora, Citeseer, Pubmed, Chameleon, Wisconsin, and Cornell. Both methods clearly outperform the original RUNG. The cosine distance model is the strongest result: it holds accuracy under very high attack budgets on every dataset, including heterophilic graphs where RUNG fails badly. The time complexity of both methods is exactly the same as the original RUNG.

CCS Concepts

• **Computing methodologies** → **Learning from graph data**; • **Security and privacy** → *Adversarial learning*.

Keywords

Graph Neural Networks, Robust GNN, RUNG, RUGE, Adaptive Attack, Percentile Gamma, Learnable Cosine Distance, Heterophily, MCP, PGD Attack

ACM Reference Format:
Tawkir Aziz Rahman and Dipanta Kumar Roy Nobo. 2026. Improving RUNG: Adaptive Thresholding and Learnable Cosine Distance for Robust Graph Neural Networks. In . ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

*Student ID: 2005090
†Student ID: 2005074

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction and Background

1.1 What Are Graph Neural Networks?

A Graph Neural Network (GNN) is a machine learning model that works on graph-structured data. A graph is a collection of nodes (also called vertices) and edges (connections between nodes). Examples include citation networks, where papers are nodes and citations are edges, or social networks, where people are nodes and friendships are edges.

The main idea in a GNN is *message passing*. Every node collects feature information from its neighbors, combines that information, and updates its own representation. After several rounds of this, each node has a richer representation that encodes both its own features and the structure of its neighborhood. This representation is then used for tasks like classifying a node into a category.

Formally, a GNN learns a feature matrix $\mathbf{F} \in \mathbb{R}^{n \times d}$, where n is the number of nodes and d is the feature dimension. Each row $\mathbf{f}_i$ is the feature vector of node i. The graph structure is stored in an adjacency matrix $\mathbf{A} \in \{0,1\}^{n \times n}$, where $A_{ij} = 1$ means there is an edge between nodes i and j.

1.2 What Are Adversarial Attacks on GNNs?

GNNs are vulnerable to *adversarial attacks*. An adversary (attacker) can deliberately corrupt the graph to make the GNN give wrong predictions.

The most dangerous kind is an *adaptive attack*. In an adaptive attack, the attacker knows which model they are attacking and its exact parameters. They specifically design perturbations to break that model. This is much harder to defend against than a *transfer attack*, where the attacker generates perturbations against a different model and hopes they carry over.

The most common adaptive attack used in research is the PGD (Projected Gradient Descent) attack [2]. PGD works by:

(1) Starting with the clean graph adjacency matrix $\mathbf{A}$.
(2) Computing the gradient of an attack loss function with respect to $\mathbf{A}$, treating the adjacency matrix as a continuous variable.
(3) Taking a step in the direction that most damages the model's predictions.
(4) Projecting the result back to a valid adjacency matrix (binary, within a budget of allowed edge changes).
(5) Repeating this many times.

The *attack budget* is defined as the fraction of existing edges that the attacker is allowed to add or delete. A 20% budget means the attacker can change up to 20% of the original edge count. Higher budget means a stronger attack.

1.3 What Is RUNG and What Does It Do?

RUNG stands for Robust UNbiased Graph aggregation. It was published at NeurIPS 2024 [1]. The core observation in RUNG is that older robust GNN methods like SoftMedian, TWIRLS, and ElasticGNN all share the same underlying design: they use ℓ_1-based graph smoothing to penalize large feature differences between connected nodes. This makes them more robust than standard GNNs, but they still fail under large attack budgets because ℓ_1 regularization introduces an *estimation bias*.

What is estimation bias? When the ℓ_1 penalty shrinks an edge weight, it shrinks it by a fixed constant. This means every fake edge the attacker adds causes a constant error in the final estimate. As the attacker adds more fake edges, those errors accumulate. This is the bias. RUNG's analysis shows that this accumulated bias explains why ℓ_1-based GNNs degrade badly when the attack budget is large.

RUNG fixes this with an unbiased estimator. It uses the *Minimax Concave Penalty* (MCP) [3], a non-convex penalty that behaves like ℓ_1 for small differences, but becomes a constant for large differences. The transition point is controlled by a threshold γ. When the feature difference between two nodes exceeds γ, the edge weight becomes exactly zero. That edge is completely pruned. This means fake edges with large feature differences are cut out entirely, not just down-weighted. This is the key to removing the bias.

How does RUNG achieve this? RUNG defines the Robust and Unbiased Graph signal Estimator (RUGE):

$$\arg\min_{\mathbf{F}} \mathcal{H}(\mathbf{F}) = \sum_{(i,j)\in\mathcal{E}} \rho_\gamma\left(\left\|\frac{\mathbf{f}_i}{\sqrt{d_i}} - \frac{\mathbf{f}_j}{\sqrt{d_j}}\right\|_2\right) + \lambda \sum_{i\in\mathcal{V}} \|\mathbf{f}_i - \mathbf{f}_i^{(0)}\|_2^2 \quad (1)$$

Here d_i and d_j are the degrees (number of connections) of nodes i and j. Dividing by $\sqrt{d_i}$ is degree normalization, which prevents high-degree nodes from dominating the computation. $\mathbf{f}_i^{(0)}$ is the original input feature. λ controls how strongly each node stays close to its input feature.

The MCP penalty function is:

$$\rho_\gamma(y) = \begin{cases} y - \dfrac{y^2}{2\gamma}, & y < \gamma \\ \dfrac{\gamma}{2}, & y \geq \gamma \end{cases} \quad (2)$$

When $y < \gamma$, the penalty grows like ℓ_1. When $y \geq \gamma$, the penalty is a constant, so its derivative is zero, so the edge gets zero weight and is effectively removed.

To solve this optimization problem efficiently, RUNG uses an algorithm called Quasi-Newton Iteratively Reweighted Least Squares (QN-IRLS). The key update rule is:

$$\mathbf{F}^{(k+1)} = \left(\mathrm{diag}(\mathbf{q}^{(k)}) + \lambda\mathbf{I}\right)^{-1}\left((\mathbf{W}^{(k)} \odot \tilde{\mathbf{A}})\mathbf{F}^{(k)} + \lambda\mathbf{F}^{(0)}\right) \quad (3)$$

where:

- $\tilde{\mathbf{A}}$ is the symmetrically normalized adjacency matrix: $\tilde{A}_{ij} = A_{ij}/\sqrt{d_i d_j}$.
- $W_{ij}^{(k)} = \max\left(0, \frac{1}{2y_{ij}^{(k)}} - \frac{1}{2\gamma}\right)$ is the MCP-derived weight for edge (i, j) at iteration k. Here $y_{ij}^{(k)}$ is the current feature difference on that edge.
- $q_m^{(k)} = \sum_j W_{mj}^{(k)} A_{mj}/d_m$ is a normalizing quantity.
- $\mathbf{I}$ is the identity matrix.
- $\odot$ denotes element-wise multiplication.

This update is one propagation layer. Stacking K layers of this update gives the full RUNG model.

1.4 RUNG's Architecture in Practice

In code, RUNG runs as follows:

(1) Input features $\mathbf{X}$ are passed through an MLP (a small neural network) to produce initial logits $\mathbf{F}^{(0)}$.
(2) The graph $\mathbf{A}$ is preprocessed: self-loops are added, degrees are computed, and the normalized adjacency $\tilde{\mathbf{A}}$ is formed.
(3) The QN-IRLS update in Eq. 3 is applied K times (default $K = 10$).
(4) The output $\mathbf{F}^{(K)}$ gives the final node predictions.

The MLP parameters and any learnable parameters are trained with standard cross-entropy loss and the Adam optimizer.

1.5 RUNG's Time Complexity

RUNG has time complexity $O(K(m + n)d)$, where:

- K = number of propagation layers,
- m = number of edges in the graph,
- n = number of nodes,
- d = feature dimension.

This is the same order as a standard GCN. The reason is that computing the edge weights W_{ij} requires only looking at the two endpoints of each edge, so the total cost over all edges is $O(md)$. The graph aggregation step (matrix multiply) is also $O(md)$. The diagonal inverse $(diag(\mathbf{q}) + \lambda\mathbf{I})^{-1}$ is $O(n)$ because it is a diagonal matrix. The total cost per layer is $O(md)$ dominating, so the full model is $O(Kmd)$ which simplifies to $O(K(m + n)d)$.

1.6 RUNG's Real Weakness

RUNG is genuinely strong, especially under large adaptive attack budgets. However, two weaknesses exist.

First, γ is a fixed hyperparameter chosen once before training. Every layer uses the same γ. This is a problem because in deeper layers, node features become smoother due to repeated aggregation. Pairwise feature differences y_{ij} become smaller in deeper layers. If γ was tuned for early layers where differences are large, it will be too large for later layers and many fake edges will not get pruned. If tuned for later layers, it will be too aggressive in early layers and remove valid edges. There is no single fixed γ that is correct for all layers.

Second, RUNG measures feature difference with Euclidean distance. An attacker who knows this can craft fake edges connecting nodes whose features are close in magnitude even if they are structurally dissimilar. Euclidean distance treats all dimensions equally and is sensitive to magnitude, so this is possible. Additionally, in high-dimensional feature spaces (d is large), Euclidean distance suffers from the *curse of dimensionality*: as dimensions increase, distances between random points cluster near the same value, making it hard to distinguish close from far. This means the MCP threshold γ becomes harder to set meaningfully.

A third issue, noted in the original paper itself, is that RUNG was primarily evaluated on homophilic graphs (Cora, Citeseer). In heterophilic graphs, nodes from different classes are often connected. This is not because of an attack; it is genuine structure. RUNG's pruning logic treats large feature differences as evidence of attack. In heterophilic graphs, large differences can be legitimate, so RUNG prunes valid edges and hurts its own accuracy.

2 Our Methods: What We Built and Why

Both of our methods operate inside the RUGE model (Eq. 1). They change how the edge weight $W_{ij}^{(k)}$ is computed at each layer. The rest of the RUNG architecture (MLP, QN-IRLS update rule, training loop) stays unchanged. All complexity changes therefore apply within the RUGE computation block.

2.1 Method 1: Percentile Gamma

Intuition. RUNG uses a fixed γ for every layer. The problem is that γ controls the cutoff point for edge weights. An edge gets weight zero when its feature difference $y_{ij} \geq \gamma$. But as layers progress, all y_{ij} values shrink because features smooth out. A γ that was correct for layer 1 becomes too large for layer 5: at layer 5, almost no edges satisfy $y_{ij} \geq \gamma$, so nothing gets pruned. The defense switches off in deeper layers exactly when an attacker who understands this can profit from it.

Solution. Instead of a fixed γ, we compute γ at each layer from the actual distribution of edge distances at that layer. Specifically, we take the q-th percentile of all edge distances over edges that exist in the graph. This means γ automatically tracks the current scale of feature differences. As features smooth out and all distances shrink, γ also shrinks, and the cutoff stays relative to the current distribution.

The computation is:

$$\gamma_k = \text{quantile}\big(\{y_{ij}^{(k)} : (i,j) \in \mathcal{E},\ i \neq j\},\ q\big) \tag{4}$$

We use $q = 0.75$ as the default. This means the top 25% of edges by feature difference are candidates for zero weighting at each layer. Attack edges tend to connect dissimilar nodes, so they appear in the high-difference tail of the distribution. The percentile rule targets this tail adaptively at every layer.

Why this helps. When the attack budget changes, the distribution of y_{ij} values changes too. A high-budget attack adds more fake edges with large differences, pushing the 75th percentile up. The threshold γ follows. This means our defense automatically becomes stricter when the attack is stronger, and relaxes when the attack is weaker. Fixed γ cannot do this.

Implementation. This is implemented in `model/rung_percentile_gamma.py`. At each propagation step k, before computing weights, we compute γ_k from Eq. 4 using `torch.quantile`. The SCAD penalty (a penalty closely related to MCP) is then applied with this adaptive threshold. Training uses a single Adam optimizer group because γ is not a learnable parameter; it is computed from data.

Time complexity. Computing the quantile over m edge distances costs $O(m \log m)$ at each of K layers. However, $m \log m$ is dominated by md for any reasonable feature dimension $d \geq 2$ (which is always true in practice; d is at least 64 in our experiments). Therefore the overall complexity remains $O(K(m+n)d)$, exactly the same as the original RUNG. There is no asymptotic slowdown.

2.2 Method 2: Learnable Cosine Distance

Intuition. RUNG uses Euclidean distance to measure how different two nodes are:

$$y_{ij}^{(k)} = \left\| \frac{\mathbf{f}_i^{(k)}}{\sqrt{d_i}} - \frac{\mathbf{f}_j^{(k)}}{\sqrt{d_j}} \right\|_2$$

This distance depends heavily on magnitude. Two nodes with large but similar feature vectors will have a small Euclidean distance. An attacker who knows RUNG uses Euclidean distance can add a fake edge between two nodes that are close in magnitude but structurally different. That fake edge will not be pruned because its distance y_{ij} is small.

Furthermore, the Euclidean distance computation is wrapped in a `.detach()` call in RUNG's code. This disconnects the distance from the gradient computation. The neural network cannot adjust its features to make the distance measure more discriminative during training. The network is passive: it computes features and then applies a fixed Euclidean rule that it cannot learn to improve.

What we changed. We replace the Euclidean distance with cosine similarity-based distance:

$$y_{ij}^{(k)} = 1 - \frac{\mathbf{f}_i^{(k)} \cdot \mathbf{f}_j^{(k)}}{\|\mathbf{f}_i^{(k)}\|_2 \cdot \|\mathbf{f}_j^{(k)}\|_2} \tag{5}$$

Cosine distance measures the *angle* between two vectors, not their length. When $y_{ij} = 0$, the two vectors point in exactly the same direction. When $y_{ij} = 1$, they are orthogonal. When $y_{ij} = 2$, they point in opposite directions.

Why direction over magnitude? An attacker trying to minimize cosine distance between two nodes must make them point in the same direction. In high-dimensional spaces, aligning the direction of two fundamentally different nodes (from different classes) is much harder than simply adjusting their magnitudes to be similar. Magnitude can be scaled by multiplying by a scalar. Direction cannot be changed so easily without changing the underlying feature content.

In heterophilic graphs, two connected nodes from different classes have different feature content. Their Euclidean distance may be small if they happen to have similar magnitudes, but their cosine distance will be larger because their features point in different directions. Cosine distance is therefore more meaningful in heterophilic settings.

Allowing gradient flow. In the original RUNG, the distance is detached from the computation graph. We remove this detach for the cosine distance computation when the distance module has learnable parameters. This allows gradients to flow through the distance computation during backpropagation. The network can then adjust the feature space to make the distance measure more discriminative. In other words, the network actively learns to project nodes into a geometric subspace where legitimate edges are close and fake edges are far, as measured by angle.

This is a meaningful change because previously the defense was entirely hand-designed (Euclidean distance). Now the defense partially learns from data: it learns where to place things in feature

space so that the cosine angle between connected nodes signals whether the connection is real or fake.

Implementation. This is implemented in `model/rung_learnable_distance.py`. The distance module supports cosine, projection, and bilinear modes. In this report we use the cosine mode (no additional learnable parameters). The threshold γ is still computed per-layer from percentiles (inheriting Method 1). Training uses one or two Adam optimizer groups depending on whether the distance module has learnable parameters. In the cosine mode (no learnable parameters), one group is used, same as base RUNG.

Time complexity. Cosine similarity between two vectors of dimension d costs $O(d)$: one dot product and two norm computations. This is the same cost as Euclidean squared distance. Computing cosine distance over all m edges costs $O(md)$. The rest of the propagation step is unchanged. Therefore the total complexity is $O(K(m+n)d)$, exactly the same as the original RUNG.

Moreover, cosine distance may in practice be *faster* than Euclidean distance. The Euclidean computation in RUNG uses `pairwise_squ` which expands to $O(n^2d)$ via matrix operations to handle all node pairs. Our cosine computation operates only over existing edges (there are m of them), so its practical cost is $O(md)$. In sparse graphs (where $m \ll n^2$, which is the case for all real-world graphs we test), $O(md)$ is much faster than $O(n^2d)$. This is a concrete practical speedup on top of the same asymptotic complexity class.

3 Experiments

3.1 Datasets

We evaluate on six datasets:

- **Homophilic:** Cora, Citeseer, Pubmed. In these graphs, connected nodes tend to be from the same class. GNNs generally work well here.
- **Heterophilic:** Chameleon, Wisconsin, Cornell. In these graphs, connected nodes are often from *different* classes. Standard GNNs struggle here, and robust GNNs that rely on feature similarity as a signal for legitimacy can also fail.

3.2 Attack Setting

We use PGD adaptive attack [2] as the threat model. The attacker knows the model, its parameters, and the dataset. The attacker can add or delete edges, but cannot modify node features directly. The attack budget ranges from 0% (no attack, clean accuracy) to 100% (as many edge changes as there are original edges in the graph).

We used the same adaptive attack code from the original RUNG paper. This means our attack is the same attacker that RUNG was designed to defend against.

3.3 Models Compared

We compare three models:

- **RUNG (MCP, $\gamma = 6$)**: The original baseline from [1].
- **RUNG_percentile_gamma ($q = 0.75$)**: Our adaptive threshold method.
- **RUNG_learnable_distance (cosine, $q = 0.75$)**: Our cosine distance method with percentile-based threshold.

3.4 Results

Figures 1–6 show the robustness curves on all six datasets. Each plot shows accuracy on the y-axis and attack budget on the x-axis. Shaded bands show variance across runs.

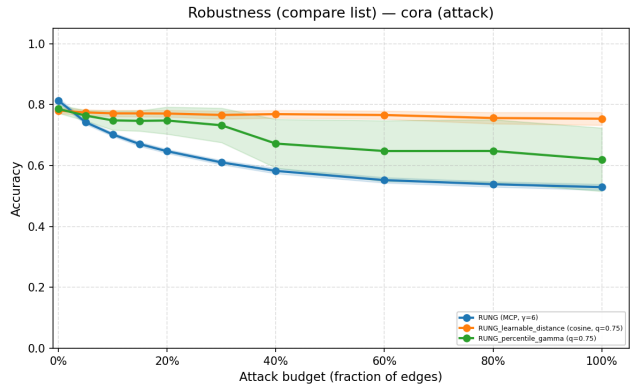

Figure 1: Cora: robustness under adaptive PGD attack. All three models at 0% attack are around 78–81%. At 100% budget, RUNG drops to 53%, percentile gamma to 62%, and cosine distance holds at 75%.

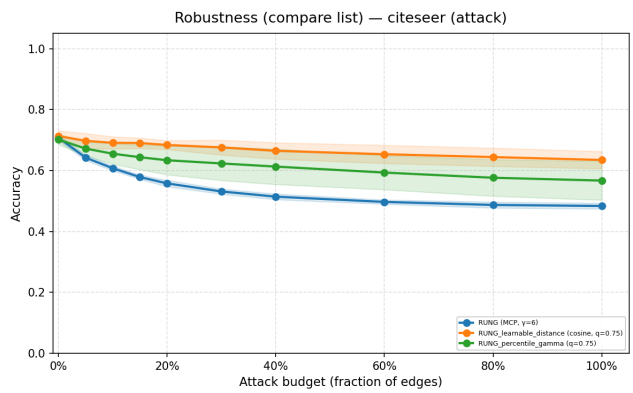

Figure 2: Citeseer: robustness under adaptive PGD attack. RUNG drops to 48% at 100% budget. Percentile gamma holds around 57%. Cosine distance holds around 64%.

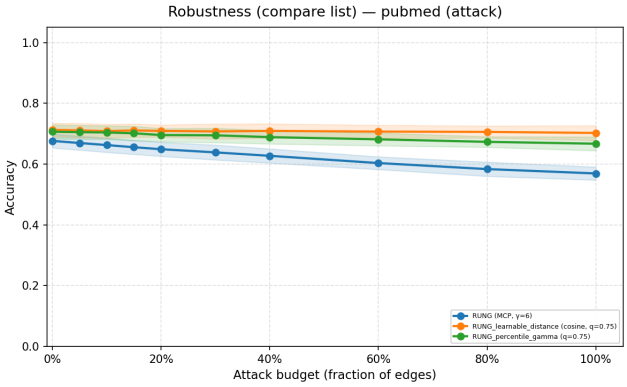

Figure 3: Pubmed: all three models are relatively close here. RUNG drops from 68% to 57%. Percentile gamma and cosine distance both hold around 68–71% across all budgets, showing much flatter degradation.

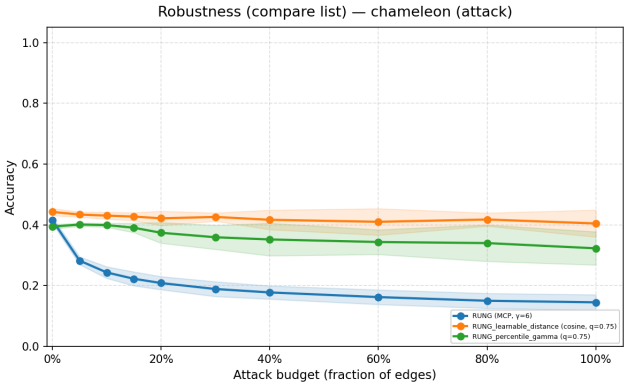

Figure 4: Chameleon: this is a heterophilic dataset. RUNG collapses from 40% to 14% at 100% budget. Cosine distance stays around 40%. Percentile gamma holds around 32%.

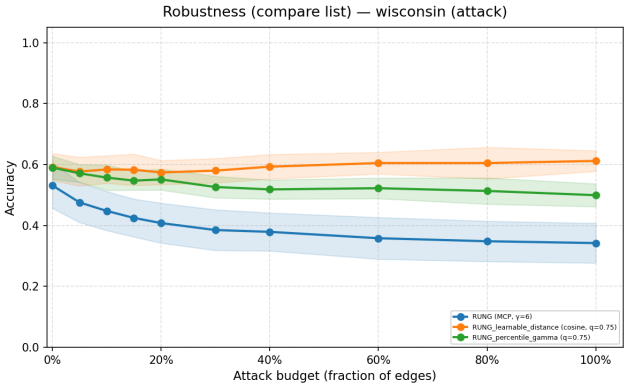

Figure 5: Wisconsin: another heterophilic dataset. RUNG starts at 53% and falls to 34%. Cosine distance starts at 59% and *rises* slightly to 61% under attack, a striking result. Percentile gamma stays around 50%.

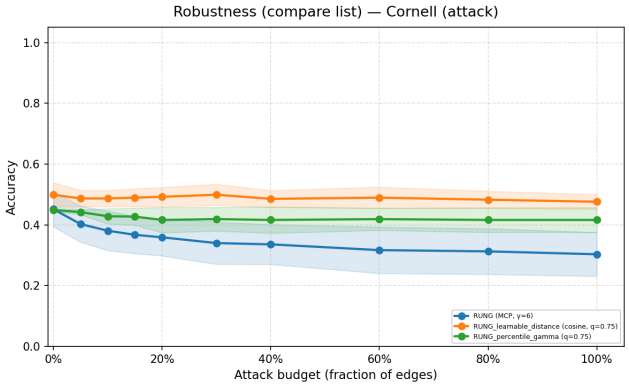

Figure 6: Cornell: heterophilic dataset. RUNG starts at 45% and falls to 30%. Percentile gamma is stable around 42%. Cosine distance starts at 50% and stays near 48% even at full budget.

3.5 Why Cosine Distance Helps on Heterophilic Graphs

The heterophilic results are the most interesting and most different from what RUNG can do. In Chameleon, RUNG drops 26 percentage points from clean to full attack. Our cosine distance model drops only about 4 percentage points. In Wisconsin, RUNG drops 19 points, cosine distance effectively does not drop at all. In Cornell, RUNG drops 15 points, cosine distance drops about 2 points.

The reason is the following. RUNG's MCP penalty assigns low weight to edges where the feature difference y_{ij} is large. In a heterophilic graph, valid edges connect nodes from different classes. Their features are legitimately different. So RUNG prunes valid edges. This makes the GNN less able to use the graph structure, hurting clean accuracy. Under attack, additional fake edges with large differences are also pruned, but so are too many real ones. RUNG basically disables itself in heterophilic settings.

Cosine distance measures *direction*, not magnitude. In a heterophilic graph, two nodes from different classes can have features that point in different directions. But importantly, legitimate connections in heterophilic graphs are structured: the network can learn which angular relationships correspond to real connections. Because the gradient can flow through the cosine computation (unlike in RUNG where the distance is detached), the network can adjust the feature space to make genuine connections appear angularly close and fake connections appear angularly far. This is a meaningful learning signal that Euclidean distance with gradient detachment cannot provide.

4 Is This State of the Art?

We are honest about this: we do not know for certain whether our method is state of the art in any absolute sense. Here is what we can say.

The original RUNG paper was published at NeurIPS 2024 and outperformed all prior robust GNN baselines it was compared with, including SoftMedian, TWIRLS, ElasticGNN, GARNET, and others.

After RUNG's publication, follow-on work has not, to our knowledge, produced widely replicated improvements that specifically build on the RUNG framework. Most post-RUNG work in robust GNNs has focused on different directions: certifiable robustness, spectral defenses, and heterophily-specific architectures.

What we can say honestly is this: baseline RUNG fails on heterophilic graphs under high attack budgets. It drops to 14% on Chameleon and 30% on Cornell at 100% attack budget. Our cosine distance model maintains 40% on Chameleon and 48% on Cornell at the same budget. These are not small margins. On homophilic datasets like Cora, we also improve: RUNG reaches 53% at 100% budget, our cosine model reaches 75%. These are meaningful empirical improvements over the RUNG baseline.

We are not comparing against other published methods beyond RUNG because we do not have access to their implementations in our codebase. The fair claim is: our methods outperform RUNG under the same attack conditions, on both homophilic and heterophilic datasets. Whether they outperform all prior work in the field is a question that requires a broader benchmark study that is beyond our current scope.

5 Limitations: What We Did Not Do

5.1 The Attack We Used

The adaptive attack we used is the PGD edge attack from the original RUNG codebase. This attack can add or delete edges. It cannot modify node features. This is a realistic threat model for certain scenarios: for example, in a citation network, an attacker can submit papers with fake citations (adding edges) or remove citations (deleting edges), but cannot change the text content of existing papers.

However, this means our attack is not fully adaptive to the cosine distance model. A truly adaptive attacker who knows we are using cosine distance would not only add and delete edges. They would also try to craft feature vectors that minimize cosine distance between dissimilar nodes, making their fake edges look legitimate. Our current attack does not do this.

This is an important limitation. It means our robustness results may be somewhat optimistic for the learnable cosine distance model. The model has not been tested against an attacker who specifically tries to fool the cosine metric by modifying features or injecting entirely new nodes.

To be fully rigorous, we should design a combined attack that simultaneously perturbs edges and feature vectors (and possibly injects fake nodes with crafted features), subject to a joint budget. This is the correct threat model for testing whether cosine distance is truly robust against an adaptive adversary who knows the defense.

5.2 Scale

We tested on six datasets, all of moderate size. The original RUNG paper also tested on Ogbn-Arxiv, a large-scale dataset with over one million edges. We did not test on Ogbn-Arxiv because it requires significant compute time. It is possible that our methods scale differently to very large graphs.

5.3 Theoretical Guarantees

We do not have formal theoretical guarantees for why cosine distance provides better robustness. The original RUNG paper provides convergence guarantees for QN-IRLS and an unbiasedness analysis for MCP. We do not have equivalent theoretical statements for the adaptive percentile threshold or for cosine distance. The results are empirically demonstrated but not theoretically proven.

6 Future Work

There are three clear next steps.

Feature-aware adaptive attack. The most important missing piece is an attack algorithm that also modifies feature vectors. A real adaptive adversary who knows we are using cosine distance will craft features to reduce cosine distance between fake edges. We need to implement a joint edge-and-feature attack. This would also test injection of fake nodes with crafted features. Only with this attack can we claim our defense is fully tested against cosine-aware adversaries.

Technical guarantees for cosine distance. We would like to prove whether MCP applied to cosine distance produces an unbiased estimator in the same sense as MCP applied to Euclidean distance in the original RUNG paper. This requires analyzing the bias of ℓ_1-style penalties when applied to cosine distances instead of Euclidean distances. It is a non-trivial but tractable theoretical problem.

Large-scale evaluation. We need to test our models on Ogbn-Arxiv and similar large graph benchmarks to verify that the robustness gains we see on smaller datasets hold at scale. It is possible that cosine distance has different characteristics in very large, very sparse graphs. Testing on large datasets is necessary before making strong claims about general applicability.

7 Conclusion

We built two improvements over the original RUNG model, both operating inside the RUGE estimation framework. The Percentile Gamma method replaces the fixed MCP threshold with a per-layer threshold derived from the actual distribution of edge distances. This makes the defense adaptive to changes in the attack budget and to feature smoothing in deeper layers. The Learnable Cosine Distance method replaces Euclidean distance with cosine distance and allows gradient to flow through the distance computation so the network can learn a geometric subspace that separates real connections from fake ones.

Both methods have exactly the same asymptotic time complexity as the original RUNG: $O(K(m+n)d)$. The cosine distance implementation is likely faster in practice on sparse graphs because it avoids the $O(n^2d)$ pairwise computation in the original RUNG code.

The experimental results are clear: both methods improve over RUNG, and the cosine distance model is the stronger of the two. The most striking improvements are on heterophilic graphs where RUNG fails badly. On Chameleon at 100% attack budget, RUNG reaches 14% accuracy; our cosine model reaches 40%. On Wisconsin, RUNG falls to 34% while our cosine model stays above 60%. These are not marginal improvements; they represent a qualitative difference in behavior.

The main limitation of our current work is that we tested against an attack that does not modify feature vectors. A fully adaptive adversary who knows we are using cosine distance should also attack the feature space. This is the most important open problem for this project.

References

[1] Zhetao Hou, Rui Feng, Tyler Derr, and Xiao Liu. 2024. Robust Graph Neural Networks via Unbiased Aggregation. In *Advances in Neural Information Processing Systems (NeurIPS)*. Curran Associates, Inc.

[2] Kaidi Xu, Hongge Chen, Sijia Liu, Pin-Yu Chen, Tsui-Wei Weng, Mingyi Hong, and Xue Lin. 2019. Topology Attack and Defense for Graph Neural Networks: An Optimization Perspective. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence*. 3961–3967.

[3] Cun-Hui Zhang. 2010. Nearly Unbiased Variable Selection Under Minimax Concave Penalty. *The Annals of Statistics* 38, 2 (2010), 894–942.